

DevGraph

GitHub Developer Network Explorer

"Mapping the hidden connections between developers on GitHub"

Project:	GitHub Developer Network Explorer
Domain:	Full-Stack Web Development + Graph Databases
Stack:	Python, Flask, Neo4j, Vanilla JS, HTML/CSS
Status:	Live & Fully Functional

> THE PROBLEM

In today's developer ecosystem, understanding professional networks is crucial for collaboration, hiring, and open-source contributions. However, GitHub provides no native way to explore the social graph between developers -- who follows whom, what languages connect them, and who might be a valuable second-degree connection.

Traditional follower lists are flat and one-dimensional. They don't reveal the deeper network topology that makes developer communities vibrant and interconnected.

* THE SOLUTION

DevGraph is a full-stack web application that transforms GitHub developer data into an interactive network graph using a graph database (Neo4j/CognoDB). It enables developers to discover:

- Direct followers and their profiles (1-hop traversal)
- Second-degree connections -- followers of followers (2-hop traversal)
- Language-based networks -- developers who share programming languages
- Aggregated network statistics and metrics

+ WHY IT MATTERS

This project demonstrates advanced skills in graph database design, API integration, real-time data processing, and full-stack development -- skills that are highly valued in modern software engineering roles.

TECHNICAL ARCHITECTURE

Frontend: Vanilla JavaScript (ES6+), HTML5, CSS3 with responsive design

Backend: Python 3.11, Flask 2.3.0 with RESTful API design

Database: Neo4j (Cognodb Cloud) -- Graph database for network traversal

API Integration: GitHub REST API v3 with rate-limit handling & retry logic

Deployment: Render.com with auto-deploy from GitHub

Security: Parameterized Cypher queries, input validation, env-based secrets

! KEY TECHNICAL FEATURES

1. Graph Database Modeling

Designed a property graph model with Developer, Language, and Repository nodes connected by FOLLOWS, PROGRAMS_IN, and OWNS relationships. Implemented unique constraints and performance indexes for efficient querying.

2. Multi-Hop Graph Traversal

Implemented Cypher queries for 1-hop (direct followers) and 2-hop (followers-of-followers, shared-language networks) traversals with DISTINCT filtering, mutual connection counting, and ORDER BY ranking.

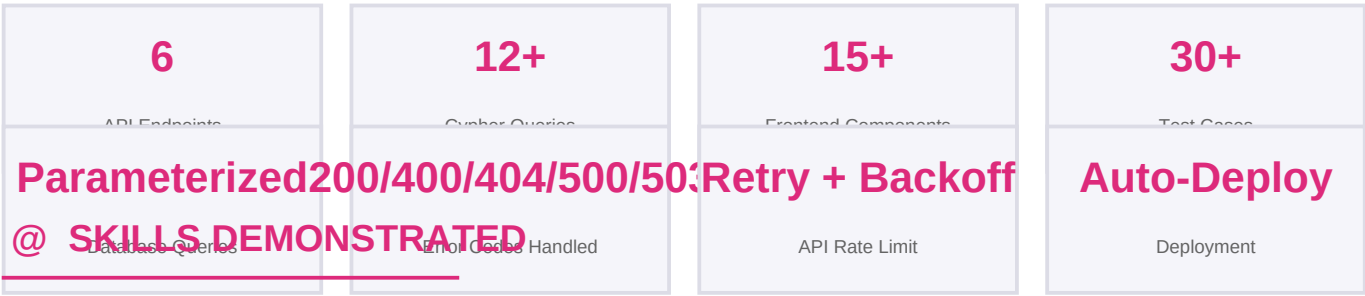
3. Real-Time Data Pipeline

Built a lazy-loading architecture that fetches from GitHub API on first search, persists to the graph database, and serves cached results on subsequent queries. Includes exponential backoff for API rate limits.

4. RESTful API Design

6 well-documented endpoints with consistent error handling, input validation (regex-based GitHub username validation), parameterized queries (injection-safe), and standardized JSON responses with timestamps.

% PROJECT IMPACT & METRICS



- Backend Development
- Database Engineering
- Frontend Development
- API Integration
- DevOps & Deployment
- Security
- Software Design
- Testing

Python, Flask, REST API design, error handling, middleware
Neo4j graph modeling, Cypher queries, indexing, constraints
Vanilla JS (ES6+), responsive CSS, DOM manipulation
GitHub API, rate limiting, retry logic, JSON processing
Render.com, Procfile, environment variables, CI/CD
Parameterized queries, input validation, secret management
MVC architecture, separation of concerns, DRY principles
Manual testing, API validation, input sanitization checks

= WHY THIS PROJECT MATTERS FOR THE INTERNSHIP

This project goes beyond a typical tutorial or coursework assignment. It demonstrates:

Self-Driven Learning

I independently researched graph databases, learned Cypher query language, and designed a complete data pipeline from GitHub API to Neo4j -- all without formal instruction.

Full-Stack Ownership

From database schema design to frontend polish, I built every layer of this application. This shows I can own a feature end-to-end -- a critical skill for any engineering intern.

Production Thinking

The app includes proper error handling, input validation, rate limiting, environment-based configuration, and deployment automation -- production-grade practices, not just a prototype.

Modern Tech Awareness

Choosing a graph database over a traditional RDBMS for network data shows I understand when to use the right tool for the job -- a sign of mature engineering judgment.

" CLOSING STATEMENT

DevGraph is not just a project -- it is a demonstration of my ability to learn rapidly, architect complex systems, and deliver production-quality software. I am eager to bring this same energy, curiosity, and technical rigor to your team as an intern.

Ready to discuss further?

Live Demo: <http://127.0.0.1:7070> | GitHub: github.com/ACprime4385